Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 2

Disciplina: Tehnici de programare

Tema: ” Lucrul în linia de comandă Linux ”

A efectuat : st. gr.SI – 266 Scorpan Andrei
A verificat : asistent universitar Cristian Gonceari

Chișinău – 2026

REGULI DE PREDARE A LUCRĂRII

Citește această pagină înainte de a începe. Nerespectarea regulilor de mai jos înseamnă că lucrarea nu poate fi identificată și nu este primită.

1. Cum se numește fișierul pe care îl trimiți

Denumirea fișierului trebuie să respecte exact acest tipar:

`L2_TP_Nume_Prenume_Grupa.docx`

Exemplu, pentru studentul Popescu Ion din grupa SI-266:

`L2_TP_Popescu_Ion_SI-266.docx`

Partea	Ce scrii	Exemplu
L2	Litera L și numărul laboratorului. La această lucrare este L2.	L2
TP	Abrevierea disciplinei, cu majuscule. Se scrie identic.	TP
Nume	Numele de familie, cu majusculă la început, fără diacritice.	Popescu
Prenume	Prenumele, cu majusculă la început, fără diacritice.	Ion
Grupa	Grupa, cu majuscule și cu cratimă.	SI-266
.docx	Extensia. Numai Word. Nu se acceptă PDF.	.docx

2. Cum trimiți lucrarea

	Regula
Adresa de email	cristian.gonceari@isa.utm.md
Subiectul emailului	L2_TP_Popescu_Ion_SI-266
	Adică exact denumirea fișierului, dar fără „.docx”. Nimic altceva în subiect.
Atașament	Un singur fișier .docx. Fără arhive, fără poze separate.
Corpul emailului	Nu este nevoie să scrii nimic.

De ce contează subiectul: cu acest subiect, toate lucrările unui laborator pot fi găsite și sortate instantaneu. Un subiect scris liber înseamnă că lucrarea ta se poate pierde printre celelalte.

3. Cum completezi acest document

1. Completează pagina de titlu. Tema lucrării este stabilită de profesor și nu se modifică. Tu înlocuiești doar textul evidențiat cu galben — grupa, numele și prenumele tău — apoi ștergi evidențierea galbenă.

2. Parcurge sarcina lucrării de la pagina următoare, pe platforma pwn.college.

3. Răspunde la toate cele 20 de întrebări, în chenarele pregătite sub fiecare întrebare. Scrie cu cuvintele tale.

4. Completează secțiunea de concluzii.

5. Salvează fișierul cu denumirea de la punctul 1 și trimite-l pe email.

SARCINA LUCRĂRII

Parcurge pe platforma pwn.college dojo-ul Linux Luminarium și rezolvă integral modulele obligatorii: Hello Hackers, Pondering Paths, Comprehending Commands, Digesting Documentation, File Globbing și Practicing Piping. Rezolvă apoi cel puțin patru dintre modulele recomandate: Shell Variables, Data Manipulation, Processes and Jobs, Untangling Users și Perceiving Permissions. Pe parcursul lucrului notează-ți comenzile exacte pe care le-ai rulat, mesajele de eroare primite și modul în care ai depășit fiecare blocaj, fiindcă întrebările din acest raport se referă la experiența ta directă, nu la definiții. Răspunde apoi, în acest document, la toate cele 20 de întrebări, cu propriile cuvinte și cu exemple concrete din sesiunile tale de lucru. Răspunsurile copiate din manuale sau de pe internet, care nu au legătură cu ceea ce ai rulat efectiv, nu se punctează.

Platforma	Cursul de parcurs	Adresa
pwn.college	Linux Luminarium	https://pwn.college/linux-luminarium/

Modulele pe care le parcurgi

Nr.	Modul	Challenge-uri	De rezolvat	Nivel
1	Hello Hackers	3	3	OBLIGATORIU
2	Pondering Paths	8	8	OBLIGATORIU
3	Comprehending Commands	15	15	OBLIGATORIU
4	Digesting Documentation	7	7	OBLIGATORIU
5	File Globbing	10	10	OBLIGATORIU
6	Practicing Piping	15	15	OBLIGATORIU
7	Shell Variables	8	8	RECOMANDAT
8	Data Manipulation	6	6	RECOMANDAT
9	Processes and Jobs	10	10	RECOMANDAT
10	Untangling Users	4	4	RECOMANDAT
11	Perceiving Permissions	8	8	RECOMANDAT
12	Chaining Commands	12	12	BONUS
13	Terminal Multiplexing	6	6	BONUS
14	Pondering PATH	5	5	BONUS
15	Silly Shenanigans	6	6	BONUS
16	Daring Destruction	5	5	BONUS

Modulele marcate OBLIGATORIU conțin materia pe care se bazează întrebările. Cele marcate RECOMANDAT și BONUS te ajută la întrebările mai dificile.

ÎNTREBĂRI

Răspunde la toate cele 20 întrebări, în chenarele de mai jos. Fiecare răspuns corect valorează 0,5 puncte.

1. La exercițiul Intro to Arguments ai rulat comanda hello mai întâi fără argument, apoi cu argumentul hackers. Explică ce face shell-ul concret cu textul pe care îl scrii la prompt înainte de a porni programul și de ce programul a reacționat diferit în cele două cazuri.

(modulul Hello Hackers)

Înainte de a porni programul, shellul analizează textul introdus: primul cuvânt este văzut ca o comandă, iar cuvintele următoare devin argumente. Astfel, când am rulat doar "hello" , programul nu a primit datele suplimentare așteptate și nu a putut oferi soluția. În schimb, când am rulat "hello hackers" , shellul a transmis direct cuvântul "hackers" către program, care la procesat și a afișat cu succes steagul.

2. La exercițiul Command History flag-ul a fost introdus în istoricul shell-ului tău. Relatează concret cum ai ajuns la el: ce ai apăsat sau ce comandă ai folosit, ce a apărut pe ecran și unde se păstrează acest istoric.

(modulul Hello Hackers)

Am deschis terminalul și am apăsat tasta "Săgeată Sus" pentru a naviga prin lista comenzilor anterioare. Pe ecran a apărut textul flagului, deoarece acesta a fost pus automat de către exercițiu direct în istoric. Acest istoric al comenzilor este salvat de către shell și este păstrat, de regulă, într-un fișier numit "history" al utilizatorului.

3. Aflându-te chiar în directorul /challenge, comanda scrisă doar cu numele programului eșuează, iar varianta cu ./ în față funcționează. Explică ce caută shell-ul în fiecare dintre cele două cazuri.

(modulul Pondering Paths)

Când scriem doar numele programului , shellul nu caută automat în directorul curent din motive de securitate, evitând astfel executarea accidentală a unor programe locale cu nume similare comenzilor de sistem. Varianta cu ./ funcționează deoarece punctul indică shell-ului să execute programul exact din locația în care ne aflăm.

4. La exercițiile implicit relative paths, from / și implicit relative path a trebuit să rulezi același program o dată din directorul / și o dată din /challenge, scriind de fiecare dată altceva. Explică de la ce punct pornește sistemul când primește o cale relativă și cum se schimbă calea pe care trebuie să o scrii atunci când te muți cu cd.

(modulul Pondering Paths)

Sistemul pornește interpretarea unei căi relative exact din directorul curent de lucru în care ne aflăm la momentul respectiv. Deoarece comanda "cd" ne modifică poziția în sistem, punctul de plecare se schimbă, ceea ce înseamnă că și calea relativă necesară pentru a ajunge la același fișier destinație trebuie modificată corespunzător.

5. La exercițiul home sweet home aveai voie să dai un argument de cel mult trei caractere. Scrie exact ce ai introdus ca argument și explică prin ce mecanism acele câteva caractere ajung să indice un fișier din directorul tău personal.

(modulul Pondering Paths)

Am introdus argumentul ~/a. Acest lucru a fost posibil datorită mecanismului de expandare al shell-ului: simbolul ~ este recunoscut și transformat automat în calea către /home/hacker. Astfel, folosind doar 3 caractere, am reușit să transmitem programului o cale absolută validă către un fișier nou.

6. Numele comenzii cat vine de la concatenare. Pe baza exercițiilor pe care le-ai rulat în Comprehending Commands, explică ce se întâmplă când îi dai mai multe fișiere ca argumente și cum se vede acest lucru în ieșirea pe care ai obținut-o.

(modulul Comprehending Commands)

Când trimiți mai multe fișiere ca argumente pentru 'cat', comanda citește conținutul fiecărui fișier în ordinea în care au fost scrise în comandă și le lipește cap la cap, trimițând rezultatul combinat direct la ecran . În ieșirea obținută, textul din primul fișier este urmat imediat de textul din al doilea fișier, fără întreruperi.

7. La exercițiul An Epic Filesystem Quest ai urmărit indicii dintr-un director în altul până la flag. Relatează concret traseul tău: ce pereche de comenzi ai repetat la fiecare pas și cum ai decis încotro mergi mai departe.

(modulul Comprehending Commands)

Pentru a găsi steagul, am folosit combinația de comenzi "ls" și "cat". Mai întâi dădeam "ls" ca să văd ce fișiere sunt în folder și să găsesc indiciul, apoi foloseam "cat" ca să citesc textul din el. Acel text îmi spunea unde trebuie să merg mai departe, așa că foloseam comanda "cd" ca să mă mut în noul director și repetam procesul.

8. La exercițiul linking files ai creat o legătură simbolică cu ln -s. Explică ce conține de fapt fișierul-legătură și ce se întâmplă în momentul în care rulezi cat pe el.

(modulul Comprehending Commands)

O legătură simbolică este practic doar un shortcut. Fișierul în sine nu conține date, ci doar calea către fișierul adevărat. Când dai comanda cat pe acest shortcut, sistemul vede că e o legătură și te redirecționează automat, afișând pe ecran direct textul din documentul original la care face referire.

9. Compară cele trei surse de documentație folosite în Digesting Documentation: man, opțiunea --help și comanda help. Spune pentru ce fel de comandă se potrivește fiecare și de ce pentru comanda-builtin din ultimul exercițiu nu ai putut folosi man.

(modulul Digesting Documentation)

‘’man’’ oferă manuale detaliate pentru programele externe instalate în sistem
‘help’ este o opțiune rapidă suportată de majoritatea programelor pentru un sumar scurt de utilizare
‘help’ este o comandă internă folosită exclusiv pentru instrucțiunile native ale shell-ului
Nu am putut folosi “man” pentru comanda din ultimul exercițiu deoarece aceasta era o comandă de tip built-in. Acestea fac parte direct din structura terminalului și nu au fișiere executabile sau manuale externe pe care “man” să le poată găsi; ele pot fi interogate doar cu funcția internă “help”

10. La exercițiul Searching For Manuals pagina de manual avea un nume generat aleator. Spune ce comandă ai folosit ca să o găsești, după ce cuvânt ai căutat și cum arăta rezultatul afișat de acea căutare.

(modulul Digesting Documentation)

Pentru a găsi manualul ascuns, am folosit comanda de căutare globală man -k. Cuvântul după care am căutat a fost “challenge” .Rezultatul afișat în terminal a fost o listă scurtă care conținea un manual cu un nume ciudat, generat random , urmat de o scurtă descriere care confirma că acela este manualul pentru provocarea respectivă.

11. Explică cine face expandarea unui tipar precum /challenge/f*: shell-ul sau programul apelat. Argumentează pe baza a ceea ce ai observat efectiv, inclusiv ce s-a întâmplat când niciun fișier nu se potrivea tiparului.

(modulul File Globbing)

Expandarea tiparelor este făcută întotdeauna de către shell înainte de pornirea programului. Acest lucru se vede clar prin faptul că argumentul este transformat în listă de fișiere de către shell, dacă niciun fișier nu se potrivea cu tiparul, shellul lăsa caracterul special neatribuit sau genera o eroare de tipul "No match found" înainte ca programul să apuce să ruleze vreodată.

12. Compară folosirea unui glob cu folosirea completării prin tasta Tab pentru a ajunge la același fișier. Spune în ce situație concretă din modul ai preferat una și în ce situație cealaltă, și de ce completarea prin Tab este considerată mai sigură.

(modulul File Globbing)

Am folosit globbing-ul atunci când a trebuit să filtrez rapid mai multe fișiere după o regulă comună . În schimb, completarea cu Tab a fost varianta mai bună când aveam nevoie de un singur fișier cu un nume lung sau când exercițiul bloca tastarea manuală. Tab-ul este considerat o metodă mult mai sigură pentru că

vezi exact pe ecran numele fișierului înainte să dai Enter. Astfel, eviți accidentele în care o steluță pusă prea general ar putea afecta din greșeală și alte fișiere de care nu voiai să te atingi

13. Descrie ce se întâmplă concret cu textul dintre două programe legate prin pipe, pe exemplul pe care l-ai rulat tu la exercițiul Grepping live output. Precizează de unde citește al doilea program și dacă textul mai trece prin vreun fișier de pe disc.

(modulul Practicing Piping)

Când am folosit pipe | la exercițiul cu Grep, textul nu a mai atins deloc hard disk-ul. Primul program și-a vărsat output-ul direct în memoria RAM, sub forma unui flux continuu, iar al doilea program a citit din zbor exact din acea "țeavă" virtuală. Astfel, am scutit complet crearea unui fișier intermediar inutil pe disc.

14. Ai redirecționat ieșirea programului /challenge/run într-un fișier folosind operatorul > și totuși pe ecran au continuat să apară mesaje. Explică de ce au apărut acele mesaje și ce ai scris ca să le capturezi și pe ele.

(modulul Practicing Piping)

Simbolul > prinde doar ieșirea standard. Mesajele care continuau să apară pe ecran erau trimise prin canalul de erori, care ocolește redirectarea clasică. Ca să le capturez și pe acelea, am folosit construcția 2>&1, forțând canalul de erori să se unifice cu cel principal înainte de a trimite datele mai departe

15. Modulul îți cere mai întâi să salvezi ieșirea într-un fișier și apoi să o filtrezi cu grep, iar la exercițiul următor să filtrezi direct prin pipe. Compară cele două metode și spune în ce situație reală ai alege salvarea în fișier și în ce situație pipe-ul direct.

(modulul Practicing Piping)

Aș folosi salvarea în fișier atunci când extrag log-uri masive pe care trebuie să le analizez de mai multe ori ulterior; așa nu mai stresez sistemul rulând comanda sursă de fiecare dată. În schimb, prefer pipe-ul direct când am nevoie de un filtru rapid, pe loc, și nu vreau să umplu partiția cu zeci de fișiere temporare. Este mult mai eficient și ține mediul de lucru curat.

16. La exercițiul Exporting Variables ai setat o variabilă, iar programul lansat de tine nu o vedea până când nu ai folosit export. Explică diferența dintre o variabilă de shell obișnuită și una exportată, din perspectiva procesului-copil.

(modulul Shell Variables)

O variabilă simplă este locală și rămâne strict în sesiunea curentă a shell-ului în care a fost creată, ceea ce înseamnă că un proces-copil nu o poate accesa. În schimb, o variabilă exportată este adăugată în mediul de execuție, fiind astfel transmisă și moștenită automat de orice proces-copil pornit din acea sesiune.

17. La exercițiile din Data Manipulation flag-ul îți venea stricat, cu caractere în plus, cu linii noi intercalate sau cu majusculele inversate. Alege unul dintre aceste exerciții, scrie comanda exactă pe care ai folosit-o pentru a repara ieșirea și spune cum arăta textul înainte și după.

(modulul Data Manipulation)

La un exercițiu din modulul Data Manipulation , /challenge/run îmi dădea flagul stricat, cu niște caractere în plus băgate printre litere - mai exact ^ și %, intercalate prin tot textul. Înainte arăta cam așa: p^%awn.^%%%co%lle^ge{%%%^^^^}

Ca să-l repar am folosit:

/challenge/run | tr -d '^%'

Opțiunea -d la tr șterge orice caracter din setul dat, în cazul ăsta ^ și %. Practic i-am zis să elimine ambele caractere oriunde apar în output, și a rămas doar flagul curat:

pwn.college{flag1234}

18. Ai apăsat Ctrl-Z peste un program care rula. Explică în ce stare ajunge procesul imediat după această apăsare și de ce el nu își continuă lucrul până când nu dai comanda bg sau fg.

(modulul Processes and Jobs)

Imediat după ce apeși Ctrl -Z, procesul intră într-o stare de suspendare ,practic este pus pe pauză și trimis în fundal. Sistemul îl îngheață temporar, oprindu-i orice execuție activă pentru a-ți reda ție controlul asupra terminalului, astfel încât să poți introduce alte comenzi. El rămâne blocat în această stare deoarece așteaptă un semnal de trezire; abia când introduci comanda 'bg' pentru a-l lăsa să ruleze ascuns, în background sau 'fg' pentru a-l aduce înapoi activ pe ecran, procesul primește permisiunea de a-și relua lucrul.

19. La exercițiul The SUID Bit ai adăugat bitul SUID unui program și abia apoi el a putut citi flag-ul. Explică cu ce identitate rulează un program obișnuit față de unul cu bit SUID și de ce asta schimbă rezultatul verificării drepturilor la citirea fișierului.

(modulul Perceiving Permissions)

Un program obișnuit rulează cu identitatea și permisiunile utilizatorului care îl pornește . Când are bitul SUID activat, programul împrumută automat identitatea celui care a creat fișierul . Asta schimbă rezultatul

verificării deoarece sistemul îl tratează ca pe un administrator, dându-i voie să citească flag-ul protejat, la care un cont normal nu ar avea acces în mod normal.

20. În modulul Untangling Users ai schimbat identitatea o dată cu su și o dată cu sudo. Explică ce parolă ți s-a cerut în fiecare dintre cele două cazuri și pe ce se bazează sistemul când decide dacă îți permite sau nu operația.

(modulul Untangling Users)

Când folosești ‘su’, sistemul îți cere parola contului în care vrei să te muți și îți oferă acces strict pe baza validării acesteia. În schimb, ‘sudo’ nu îți cere parola de administrator, ci se bazează pe politicile interne care verifică automat dacă utilizatorul tău are permisiunea prestabilită să execute comenzi cu drepturi speciale

CONCLUZII

Scrie 10 – 15 rânduri: ce ai învățat concret, ce ți s-a părut cel mai dificil, unde te-ai blocat și cum ai ieșit din blocaj. Evită formulările generale de tipul „lucrarea a fost utilă”.

În această lucrare am aprofundat interacțiunea cu linia de comandă în Linux, concentrându-mă pe gestionarea proceselor, redirecționări și lucrul pe fundal. Concret am învățat să identific programe ascunse folosind ‘ps’ și ‘grep’, și să opresc procesele care îmi blocau resursele utilizând comanda ‘kill’. De asemenea, am exersat suspendarea și mutarea task-urilor între planul principal și secundar, prin Ctrl-Z, ‘bg’ și ‘fg’, lucru esențial când aveam nevoie să rulez mai multe instanțe ale aceluiași program pe un singur terminal. Cel mai dificil aspect mi s-a părut lucrul cu țevile . M-am blocat complet când terminalul mi-a înghețat, deoarece un program impostor umplea țeava cu mesaje false. Am ieșit din acest impas izolând și oprind forțat acel proces fals, după care am înțeles că trebuie să rulez comanda principală în background adăugând ‘&’ la final. Astfel, mi-am eliberat tastatura pentru a putea extrage datele curate cu ‘cat’. La final, lucrul cu variabila ‘$?’ m-a ajutat să înțeleg cum pot verifica rapid din terminal dacă o operațiune a dat eroare sau s-a executat cu succes, citind codul de ieșire lăsat în urmă.

BIBLIOGRAFIE

1. pwn.college (Arizona State University), Linux Luminarium, [Online]. Disponibil: https://pwn.college/linux-luminarium/

2. pwn.college, linux-luminarium (codul sursă și descrierile modulelor și ale challenge-urilor), GitHub, [Online]. Disponibil: https://github.com/pwncollege/linux-luminarium

3. Free Software Foundation, GNU Bash Reference Manual, [Online]. Disponibil: https://www.gnu.org/software/bash/manual/bash.html

4. M. Kerrisk, The Linux man-pages project (paginile de manual ale comenzilor Linux), [Online]. Disponibil: https://man7.org/linux/man-pages/

5. W. Shotts, The Linux Command Line: A Complete Introduction, No Starch Press, [Online]. Disponibil: https://linuxcommand.org/tlcl.php

6. The Linux Foundation, Filesystem Hierarchy Standard (FHS), versiunea 3.0, [Online]. Disponibil: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html

BAREM DE NOTARE

Nota se calculează numai din răspunsurile la cele 20 de întrebări.
20 întrebări × 0,5 puncte = 10 puncte = nota 10

Cum se punctează un răspuns

Punctaj	Când se acordă
0,5 puncte	Răspuns corect şi complet: explică mecanismul, nu doar enunță rezultatul.
0,25 puncte	Răspuns parțial corect: ideea principală este bună, dar explicația este incompletă sau conține o imprecizie.
0 puncte	Răspuns greșit, lipsă, sau copiat cuvânt cu cuvânt de la alt student ori dintr-o sursă de pe internet.

Cum se transformă punctajul în notă

Răspunsuri corecte	Punctaj	Nota
20 răspunsuri corecte	10 puncte	10
18 – 19 răspunsuri corecte	9 – 9,5 puncte	9
16 – 17 răspunsuri corecte	8 – 8,5 puncte	8
14 – 15 răspunsuri corecte	7 – 7,5 puncte	7
12 – 13 răspunsuri corecte	6 – 6,5 puncte	6
10 – 11 răspunsuri corecte	5 – 5,5 puncte	5
sub 10 răspunsuri corecte	sub 5 puncte	nepromovat — lucrarea se reface